

贪心算法

姓名：李盛鹏 学号：2351136 日期：2025年5月17日

完成所有交易的初始最少钱数

给你一个二维整数数组 `transactions`，其中 `transactions[i] = [costi, cashbacki]` 表示第 `i` 笔交易的成本和返现。

你的初始钱数需要以任意顺序完成所有交易。对于每一笔交易，你必须先支付 `costi` 的钱数，然后在交易完成后得到 `cashbacki` 的钱数。如果在任何时候你的钱数小于 0，交易将无法完成。

请你返回完成所有交易的初始最少钱数。

示例

示例 1

输入：transactions = [[2,1],[5,0],[4,2]] 输出：10 解释：一开始钱数为 10，我们可以按以下顺序完成交易：

- 完成第 2 笔交易，钱数变为 $10 - 5 + 0 = 5$
- 完成第 1 笔交易，钱数变为 $5 - 2 + 1 = 4$
- 完成第 3 笔交易，钱数变为 $4 - 4 + 2 = 2$ 无法以少于 10 的钱数开始完成所有交易。

示例 2

输入：transactions = [[3,0],[0,3]] 输出：3 解释：

- 如果初始钱数为 3，我们可以按任意顺序完成交易。
- 无法以少于 3 的钱数开始完成所有交易。

解答

```
1 class Solution {
2 public:
3     long long minimumMoney(vector<vector<int>>& transactions) {
4         long long total_lose = 0;
5         int max_cost_in_earning = 0;
6         int max_cashback_in_lost = 0;
7         for (auto& t : transactions) {
8             total_lose += max(t[0] - t[1], 0); // 计算最大亏损
9             if(t[0]<t[1]) max_cost_in_earning = max(max_cost_in_earning, t[0]);
10            else max_cashback_in_lost = max(max_cashback_in_lost, t[1]);
11        }
12
13        long long res = total_lose + max(max_cashback_in_lost, max_cost_in_earning);
14        return res;
15    }
16};
```

通过 56 / 56 个通过的测试用例

Loving SvansontPq 提交于 2025.05.17 13:53

官方题解

写题解

执行用时分布

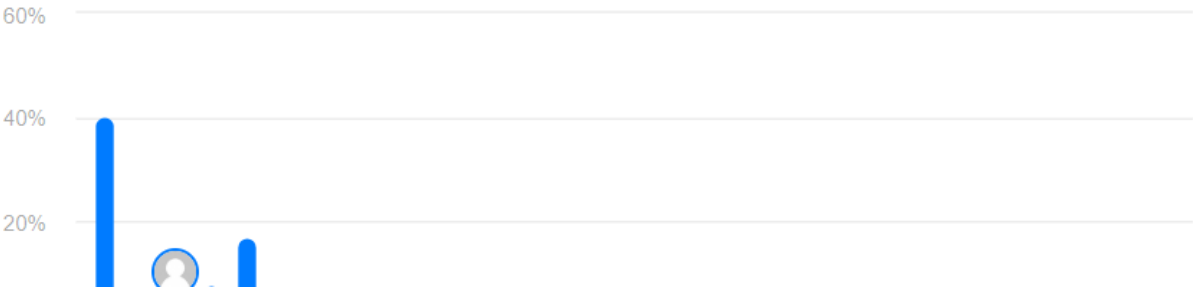


2 ms | 击败 57.59%

复杂度分析

消耗内存分布

112.20 MB | 击败 64.40%



完成所有任务的最少时间

你有一台电脑，可以同时运行无数个任务。给你一个二维整数数组 `tasks`，其中 `tasks[i] = [start_i, end_i, duration_i]` 表示第 `i` 个任务需要在闭区间 `[start_i, end_i]` 内运行 `duration_i` 个时间单位（不需要连续）。

当电脑在运行某个任务时，它可以在时间段 `[start_i, end_i]` 内的任意时间单位执行该任务，包括边界 `start_i` 和 `end_i`。但是，同一时间单位不能同时运行多个任务。

请你返回完成所有任务所需的最少时间单位数。

示例

示例 1

输入:

```
tasks = [[2,3,1],[4,5,1],[1,5,2]]
```

输出:

2

解释:

- 第一个任务在时间单位 2 运行。
- 第二个任务在时间单位 5 运行。
- 第三个任务在时间单位 1 和 3 运行（或时间单位 1 和 4）。总共需要 2 个时间单位，因此返回 2。

示例 2

输入:

```
tasks = [[1,3,2],[2,5,3],[5,6,2]]
```

输出:

4

解释:

- 第一个任务在时间单位 2 和 3 运行。
- 第二个任务在时间单位 2、4 和 5 运行。
- 第三个任务在时间单位 5 和 6 运行。总共需要 4 个时间单位，因此返回 4。

解答

```
1 class Solution {
2 public:
3     int findMinimumTime(vector<vector<int>>& tasks) {
4         int n = tasks.size();
5         sort(tasks.begin(), tasks.end(), [&](vector<int> t1, vector<int> t2){
6             return t1[1] < t2[1];
7         });
8
9         vector<int> run(tasks[n-1][1] + 1, 0); // 制作一个时间轴
10        int res = 0;
11        for(int i = 0; i < n; i++){
12            int start = tasks[i][0], end = tasks[i][1], duration = tasks[i][2];
13            duration -= accumulate(run.begin() + start, run.begin() + end + 1, 0);
14            res += max(duration, 0);
15            for(int j = end; j >= 0 && duration > 0; j--){
16                if(run[j] == 0){
17                    run[j] = 1;
18                    duration--;
19                }
20            }
21        }
22
23        return res;
24    }
25};
```

[← 全部提交记录](#)

通过 1059 / 1059 个通过的测试用例

Loving SvansontPq 提交于 2025.05.17 15:08

[官方题解](#)[写题解](#)

🕒 执行用时分布



175 ms | 击败 **13.07%**

🌟 [复杂度分析](#)

💾 消耗内存分布

74.13 MB | 击败 **13.72%**

